

Nome: Guilherme de Moraes China Costa

Data: 25/10/2025

Título: Engenharia de Testes de Software - TP1

Parte 1: Teste e Validação do Sistema de Cálculo de IMC

O código analisado foi o **CalculoIMC.java**.

Análise Estrutural Rápida

O código está bem organizado — a lógica de interação (método `programaIMC`) está separada da lógica de negócio (**`calcularPeso`** e **`classificarIMC`**). Isso é ótimo, porque facilita bastante a criação de testes unitários.

Mas há alguns pontos vulneráveis:

- **`Double.parseDouble(pScan.nextLine())`** quebra se o usuário digitar algo como "abc" ou "1,75" (com vírgula, formato brasileiro).
 - **`calcularPeso(peso, altura)`** retorna **`Infinity`** se a altura for 0.
 - O programa aceita valores negativos para peso e altura, o que obviamente gera resultados incorretos.
-

1. Teste Exploratório — Avaliação Geral

Depois de compilar (**`javac CalculoIMC.java`**) e executar (**`java CalculoIMC`**), fiz alguns testes exploratórios:

Cenário “Caminho Feliz” (Happy Path)

Input: Peso = 70, Altura = 1.75

Output:

Seu índice de massa corporal é: 22,86 kg/m²

Classificação: Saudável.

Avaliação: Tudo funcionando como esperado para entradas válidas.

Cenário de Entrada Inválida (Texto)

Input: Peso = abc

Output:

O programa quebra com a exceção:

Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"

Avaliação: Falha funcional crítica — não há nenhum tratamento para entradas não numéricas.

Cenário de Entrada Inválida (Locale/Vírgula)

Input: Peso = 70, Altura = 1,75

Output:

Mesmo erro do cenário anterior (NumberFormatException).

Avaliação: Outra falha crítica. O programa não entende o formato de número com vírgula (muito comum em português).

Cenário de Divisão por Zero

Input: Peso = 70, Altura = 0

Output:

Seu índice de massa corporal é: Infinity kg/m²

Classificação: Obesidade Grau III.

Avaliação: Falha crítica. A divisão por zero gera **Infinity**, que é classificado incorretamente como “Obesidade Grau III”.

Cenário de Valores Negativos

Input: Peso = -70, Altura = 1.75

Output:

Seu índice de massa corporal é: -22,86 kg/m²

Classificação: Magreza grave.

Avaliação: Falha funcional alta. O programa aceita um peso negativo e acaba gerando um IMC negativo — algo que obviamente não faz sentido.

Cenário com NaN (Not a Number)

Input: Peso = 0, Altura = 0

Output:

Seu índice de massa corporal é: NaN kg/m²

Classificação: Obesidade Grau III.

Avaliação: Falha crítica. O cálculo 0.0 / 0.0 gera NaN, e, como NaN não satisfaz nenhuma comparação, ele acaba caindo no else final — sendo classificado incorretamente.

Experiência do Usuário (Usabilidade)

- **Instruções:** Claras.
- **Interação:** Funciona bem no caminho feliz, mas é bem punitiva quando o usuário erra.
- **Feedback de Erro:** Praticamente inexistente. O programa simplesmente “explode” (crasha) ou mostra valores absurdos (Infinity, NaN, -22,86) sem explicar o motivo.

2. Identificação de Problemas

ID	Problema	Tipo	Prioridade	Descrição
P-001	Crash em Entrada Não Numérica	Funcional	Crítica	O programa falha com <code>NumberFormatException</code> se o usuário digitar texto (ex: "abc") ou usar vírgula (ex: "1,75").
P-002	Divisão por Zero (Altura = 0)	Funcional	Crítica	Se a altura for 0, o IMC resulta em <code>Infinity</code> e é classificado incorretamente como "Obesidade Grau III".
P-003	Cálculo NaN (0 / 0)	Funcional	Crítica	Se peso e altura forem 0, o IMC resulta em <code>NaN</code> e é classificado incorretamente como "Obesidade Grau III".
P-004	Valores Negativos Aceitos	Funcional	Alta	O sistema aceita peso e/ou altura negativos, resultando em IMC negativo e classificação incorreta ("Magreza grave").

P-005	Ausência de Feedback de Erro	Usabilidade	Alta	O usuário não é informado sobre o que errou. O programa deve validar as entradas e solicitar a correção.
P-006	Uso de dois Scanners	Técnico	Baixa	O código cria <code>pScan</code> e <code>aScan</code> , ambos lendo de <code>System.in</code> . É desnecessário e considerado má prática, embora não quebre o programa.

3. Especificação do Comportamento Esperado

O programa deve ser robusto contra todas as entradas do usuário.

- Refatoração (Lógica de Negócio):** Os métodos de lógica de negócio (`calcularPeso` e `classificarIMC`) devem ser responsáveis por validar seus próprios domínios.
 - `calcularPeso(peso, altura)`: Deve lançar `IllegalArgumentException` se `peso <= 0` ou `altura <= 0`.
 - `classificarIMC(imc)`: Deve tratar casos especiais como `Infinity` e `NaN` (talvez lançando uma exceção ou retornando "Inválido"). Nota: Se `calcularPeso` for corrigido, `classificarIMC` nunca receberá `Infinity` ou `NaN` de `calcularPeso`.
- Entrada de Dados (Método `programaIMC`):**
- Deve usar um único Scanner.
- Deve usar um loop (ex: `while(true)`) para solicitar o peso até que um valor válido seja inserido.
- Deve usar try-catch (`NumberFormatException`) para capturar erros de digitação (como "abc").
- Deve verificar se o valor é positivo (`> 0`).
- Se a entrada for inválida, deve exibir uma mensagem clara (ex: "Erro: Digite um valor numérico positivo (ex: 70.5)").
 - Deve fazer o mesmo para a altura.
- Processamento (Método `programaIMC`):**
 - O método `programaIMC` deve chamar `calcularPeso` e `classificarIMC` apenas com dados validados.

4. Partições Equivalentes para Entrada de Dados

Dividimos os testes em partições para as entradas dos métodos lógicos.

Método: calcularPeso(peso, altura)

- **Partição 1 (Peso):**
 - Válida: peso > 0 (ex: 70, 0.1)
 - Inválida (Zero): peso = 0
 - Inválida (Negativo): peso < 0 (ex: -70)
- **Partição 2 (Altura):**
 - Válida: altura > 0 (ex: 1.75, 0.5)
 - Inválida (Zero): altura = 0
 - Inválida (Negativo): altura < 0 (ex: -1.75)

Método: classificarIMC(imc)

- **Válidas:**
 - PV-MG: imc < 16.0 (ex: 15.9)
 - PV-MM: 16.0 <= imc < 17.0 (ex: 16.5)
 - PV-ML: 17.0 <= imc < 18.5 (ex: 18.0)
 - PV-S: 18.5 <= imc < 25.0 (ex: 22.0)
 - PV-SP: 25.0 <= imc < 30.0 (ex: 27.0)
 - PV-O1: 30.0 <= imc < 35.0 (ex: 32.0)
 - PV-O2: 35.0 <= imc < 40.0 (ex: 38.0)
 - PV-O3: imc >= 40.0 (ex: 45.0)
- **Inválidas (Geradas por calcularPeso defeituoso):**
 - PI-Inf: Infinity
 - PI-NaN: NaN
 - PI-Neg: imc < 0 (ex: -22.8)

5. Análise de Limites

Focamos nos valores de fronteira do método classificarIMC, que são os pontos mais prováveis de falha lógica (ex: usar < em vez de <=).

Cas o	Método Testado	Entrad a (IMC)	Objetivo do Teste	Resultado Esperado	Resultado Obtido	Status
AL-01	classificarIMC	15.99	Limite Magreza grave	"Magreza grave"	"Magreza grave"	Passo u
AL-02	classificarIMC	16.0	Limite Magreza moderad a	"Magreza moderada"	"Magreza moderada"	Passo u
AL-03	classificarIMC	16.99	Limite Magreza moderad a	"Magreza moderada"	"Magreza moderada"	Passo u
AL-04	classificarIMC	17.0	Limite Magreza leve	"Magreza leve"	"Magreza leve"	Passo u
AL-05	classificarIMC	18.49	Limite Magreza leve	"Magreza leve"	"Magreza leve"	Passo u
AL-06	classificarIMC	18.5	Limite Saudável	"Saudável"	"Saudável"	Passo u

AL-07	classificarIMC	24.99	Limite Saudável	"Saudável"	"Saudável"	Passo u
AL-08	classificarIMC	25.0	Limite Sobrepeso	"Sobrepeso"	"Sobrepeso"	Passo u
AL-09	classificarIMC	39.99	Limite Obesidade Grau II	"Obesidade Grau II"	"Obesidade Grau II"	Passo u
AL-10	classificarIMC	40.0	Limite Obesidade Grau III	"Obesidade Grau III"	"Obesidade Grau III"	Passo u

Conclusão da Análise de Limites: A lógica de classificação (classificarIMC) está correta e robusta para dados numéricos válidos. Os if (imc == X.X || ...) são redundantes, mas não causam bugs. O problema não está na lógica de classificação, mas nos dados inválidos (Infinity, NaN, Negativos) que ela recebe do calcularPeso.

6. Cobertura de Código com JaCoCo

Sugestão de Novos Casos de Teste (para 100% de cobertura de ramo): Para garantir que todos os ramos de classificarIMC sejam testados, precisamos de pelo menos 8 casos de teste, um para cada partição identificada na Tarefa 4 (AL-01 a AL-10 já cobrem isso):

```
@Test void testaMagrezaGrave() { assertEquals("Magreza grave",
CalculoIMC.classificarIMC(15.0)); }
```

```
@Test void testaMagrezaModerada() { assertEquals("Magreza moderada",
CalculoIMC.classificarIMC(16.5)); }
```

```
@Test void testaMagrezaLeve() { assertEquals("Magreza leve",
CalculoIMC.classificarIMC(17.5)); }
```

```
@Test void testaSaudavel() { assertEquals("Saudável", CalculoIMC.classificarIMC(22.0)); }
```

```
@Test void testaSobrepeso() { assertEquals("Sobrepeso",
CalculoIMC.classificarIMC(27.0)); }
```

```
@Test void testaObesidade1() { assertEquals("Obesidade Grau I",
CalculoIMC.classificarIMC(32.0)); }
```

```
@Test void testaObesidade2() { assertEquals("Obesidade Grau II",  
CalculoIMC.classificarIMC(37.0)); }
```

```
@Test void testaObesidade3() { assertEquals("Obesidade Grau III",  
CalculoIMC.classificarIMC(45.0)); }
```

Proposta de Solução (Refatoração): Para corrigir os bugs (P-002, P-003, P-004) e torná-los visíveis no JaCoCo, o método calcularPeso deve ser refatorado para incluir validações:

// Versão Refatorada

```
public static double calcularPeso(double peso, double altura) {  
    if (peso <= 0) {  
        throw new IllegalArgumentException("Peso deve ser positivo.");  
    }  
    if (altura <= 0) {  
        throw new IllegalArgumentException("Altura deve ser positiva.");  
    }  
    return peso / (altura * altura);  
}
```

Novos Casos (para 100% de cobertura do código refatorado):

```
@Test void testaPesoZero() {  
    assertThrows(IllegalArgumentException.class, () -> {  
        CalculoIMC.calcularPeso(0, 1.75);  
    });  
}
```

```
@Test void testaAlturaZero() {  
    assertThrows(IllegalArgumentException.class, () -> {  
        CalculoIMC.calcularPeso(70, 0);  
    });  
}
```


Parte 2: Quality Intelligence (QI) com Jqwik e Mockito

1. Explicação: Testes Baseados em Propriedades (PBT)

Diferença dos Testes Tradicionais: Testes tradicionais (baseados em *exemplos*, como JUnit) são reativos. Nós pensamos em um cenário (ex: Peso 70, Altura 1.75), calculamos o resultado esperado (22.86) e escrevemos um teste que verifica *apenas esse exemplo*. O problema é que só testamos o que conseguimos imaginar.

Testes Baseados em Propriedades (PBT) são proativos. Em vez de testar *exemplos*, testamos *regras* (propriedades) que devem ser verdadeiras para *qualquer* entrada. Nós definimos a regra e a ferramenta (Jqwik) gera centenas de dados (incluindo valores extremos, zeros, negativos, NaN, Infinity) para tentar *quebrar* essa regra.

Vantagens:

1. **Geração Automática de Dados:** O Jqwik pensa em casos extremos que um humano esqueceria (ex: 0.0 / 0.0).
2. **Identificação de Casos Inesperados:** É perfeito para encontrar os bugs que identificamos na Parte 1 (P-002, P-003, P-004).
3. **Maior Cobertura (Lógica):** Um teste de propriedade substitui dezenas de testes de exemplo.
4. **Contraprovações (Shrinking):** Quando o Jqwik acha uma falha (ex: com altura -1.2345e-10), ele não reporta esse número. Ele "encolhe" (shrink) o valor até encontrar o menor contraexemplo possível (ex: 0.0 ou -1.0), facilitando a depuração.

Exemplo de Propriedade para o IMC: *Propriedade:* "Para qualquer peso positivo e qualquer altura positiva, o IMC calculado deve ser *sempre* um número positivo." (O código atual falha nisso se o peso for negativo).

2. Criando Testes Baseados em Propriedades com Jqwik

```
com.xml J App.java J CalculoIMC.java 2 J AppTest.java J CalculoIMCPropertiesTest.java X J CalculoIMCExtremeTest.java
c > test > java > com > engenharia_teste > TP1 > J CalculoIMCPropertiesTest.java > CalculoIMCPropertiesTest
39 package com.engenharia_teste.TP1;
38
37 import net.jqwik.api.*;
36 import net.jqwik.api.constraints.DoubleRange;
35 import net.jqwik.api.constraints.Positive;
34 import static org.assertj.core.api.Assertions.assertThat;
33
32 import org.assertj.core.api.Assumptions;
31
30 class CalculoIMCPropertiesTest {
29
28     @Property
27     void imcNuncaDeveSerNegativo(
26         @ForAll @DoubleRange(min = 1.0, max = 300.0) double peso,
25         @ForAll @DoubleRange(min = 0.5, max = 2.5) double altura) {
24         double imc = CalculoIMC.calcularPeso(peso, altura);
23
22         assertThat(imc).isGreaterThan(0);
21     }
20
19     @Property
18     void classificacaoDeveSerMonotonica(@ForAll @Positive double imc1, @ForAll @Positive double imc2) {
17         Assumptions.assumeThat(imc1 < imc2);
16
15         String class1 = CalculoIMC.classificarIMC(imc1);
14         String class2 = CalculoIMC.classificarIMC(imc2);
13
12         java.util.Map<String, Integer> ordem = java.util.Map.of(
11             "Magreza grave", 0,
10             "Magreza moderada", 1,
9             "Magreza leve", 2,
8             "Saudável", 3,
7             "Sobrepeso", 4,
6             "Obesidade Grau I", 5,
5             "Obesidade Grau II", 6,
4             "Obesidade Grau III", 7);
3
2         assertThat(ordem.get(class1)).isLessThanOrEqualTo(ordem.get(class2));
1     }
0 }
```

3. Gerando Conjuntos Diversificados de Dados

```
src > test > java > com > engenharia_teste > TP1 > J CalculoIMCExtremeTest.java > CalculoIMCExtremeTest
28 package com.engenharia_teste.TP1;
27 import net.jqwik.api.*;
26 import static org.assertj.core.api.Assertions.assertThat;
25
24 class CalculoIMCExtremeTest {
23
22     @Provide
21     Arbitrary<Double> pesosExtremos() {
20         return Arbitraries.doubles().between(30.0, 500.0);
19     }
18
17     @Provide
16     Arbitrary<Double> alturasExtremas() {
15         return Arbitraries.doubles().between(0.5, 2.8);
14     }
13
12     @Property
11     void testIMCComValoresExtremos(
10         @ForAll("pesosExtremos") double peso,
9         @ForAll("alturasExtremas") double altura
8     ) {
7         double imc = CalculoIMC.calcularPeso(peso, altura);
6         String classificacao = CalculoIMC.classificarIMC(imc);
5
4         assertThat(imc).isPositive();
3
2         assertThat(classificacao).isNotNull().isNotEmpty();
1     }
29 }
```

4. Analisando Contraprovações Geradas pelo Jqwik

```
> test > java > com > engenharia_teste > TP1 > J CalculoIMCFailureTest.java > CalculoIMCFailureTest
17 package com.engenharia_teste.TP1;
16
15 import net.jqwik.api.*;
14 import static org.assertj.core.api.Assertions.assertThat;
13
12 class CalculoIMCFailureTest {
11
10     @Property
9     void testIMCComValoresAleatorios(@ForAll double peso, @ForAll double altura) {
8         double imc = CalculoIMC.calcularPeso(peso, altura);
7
6         String classificacao = CalculoIMC.classificarIMC(imc);
5
4         if (classificacao.equals("Obesidade Grau III")) {
3             assertThat(imc).isGreaterThanOrEqualTo(40.0);
2         }
1     }
8 }
```

5. Isolando Dependências com Mocks (Mockito)

```
pom.xml J App.java J CalculoIMC.java 2 J AppTest.java J CalculoIMCPropertiesTest.java J DashboardServiceTest.java X J DashboardService
src > test > java > com > engenharia_teste > TP1 > J DashboardServiceTest.java > DashboardServiceTest > dashboardService
16
15 import static org.mockito.Mockito.*;
14 import static org.assertj.core.api.Assertions.assertThat;
13
12 import org.junit.jupiter.api.Test;
11 import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.InjectMocks;
9 import org.mockito.Mock;
8 import org.mockito.junit.jupiter.MockitoExtension;
7
6 @ExtendWith(MockitoExtension.class)
5 class DashboardServiceTest {
4
3     @Mock
2     CalculadoraIMCService calculadoraMock;
1
18     @InjectMocks
1     DashboardService dashboardService;
2
3     @Test
4     void testMensagemQuandoEstiverSaudavel() {
5         when(calculadoraMock.calcularPeso(70, 1.75)).thenReturn(22.8);
6         when(calculadoraMock.classificarIMC(22.8)).thenReturn("Saudável");
7
8         String mensagem = dashboardService.getMensagemSaude(70, 1.75);
9
10        assertThat(mensagem).isEqualTo("Parabéns! Você está saudável.");
11
12        verify(calculadoraMock).calcularPeso(70, 1.75);
13        verify(calculadoraMock).classificarIMC(22.8);
14    }
15
16    @Test
17    void testMensagemQuandoEstiverComSobrepeso() {
18        when(calculadoraMock.calcularPeso(90, 1.75)).thenReturn(29.4);
19        when(calculadoraMock.classificarIMC(29.4)).thenReturn("Sobrepeso");
20
21        String mensagem = dashboardService.getMensagemSaude(90, 1.75);
22
23        assertThat(mensagem).isEqualTo("Atenção! Risco de sobrepeso.");
24    }
25
26    @Test
27    void testMensagemQuandoEstiverComMagreza() {
28        when(calculadoraMock.calcularPeso(40, 1.75)).thenReturn(13.1);
29        when(calculadoraMock.classificarIMC(13.1)).thenReturn("Magreza grave");
30
31        String mensagem = dashboardService.getMensagemSaude(40, 1.75);
32
33        assertThat(mensagem).isEqualTo("Atenção! Risco de desnutrição.");
34    }
35 }
```

6. Criando Testes Baseados em Propriedades para Valores Específicos

```
J DashboardServiceTest.java J DashboardService.java J CalculadoraIMCService.java J CalculoIMCExtremeTest.java J CalculoIMC
src > test > java > com > engenharia_teste > TP1 > J CalculadoraIMCLimitesTest.java > CalculadoraIMCLimitesTest
41 package com.engenharia_teste.TP1;
40
39 import net.jqwik.api.*;
38 import static org.assertj.core.api.Assertions.assertThat;
37
36 class CalculadoraIMCLimitesTest {
35
34     @Example
33     void testeLimiteExatoMagrezaGrave() {
32         double imc = 15.99;
31         String classif = CalculoIMC.classificarIMC(imc);
30         assertThat(classif).isEqualTo("Magreza grave");
29     }
28
27     @Example
26     void testeLimiteExatoMagrezaModerada() {
25         double imc = 16.0;
24         String classif = CalculoIMC.classificarIMC(imc);
23         assertThat(classif).isEqualTo("Magreza moderada");
22     }
21
20     @Example
19     void testeLimiteExatoSaudavel() {
18         double imc = 18.5;
17         String classif = CalculoIMC.classificarIMC(imc);
16         assertThat(classif).isEqualTo("Saudável");
15     }
14
13     @Example
12     void testeLimiteExatoSobrepeso() {
11         double imc = 25.0;
10         String classif = CalculoIMC.classificarIMC(imc);
9         assertThat(classif).isEqualTo("Sobrepeso");
8     }
7
6     @Example
5     void testeLimiteExatoObesidadeGrauIII() {
4         double imc = 40.0;
3         String classif = CalculoIMC.classificarIMC(imc);
2         assertThat(classif).isEqualTo("Obesidade Grau III");
1     }
42 }
```

Explicação da Escolha dos Valores: Esses valores (16.0, 18.5, 25.0, 40.0) foram escolhidos por serem os pontos de fronteira exatos entre as classificações. Erros de lógica (como usar < em vez de <= ou a ordem errada dos ifs) são mais prováveis de aparecer nesses limites. Como vimos na Análise de Limites (Parte 1, Tarefa 5), a lógica do código original está correta e esses testes passarão, validando a robustez da classificação.

Como usei I.A no projeto (esclarecimento)

Usei I.A para fazer a geração desse relatório, para acelerar o processo. Mas todas as explorações e codificação foram feitas sem auxílio.